

Implementation Statement

Nguyen Huy An Khanh

COMP2050 Programming Project

Project Overview

This statement describes how the implementation work for the Minecraft sword PvP agent was performed. The submitted code implements a Monte Carlo Tree Search based combat agent, a local prismarine-physics duel simulator, experiment logging, and report-material generation.

Existing Libraries and External Sources

The project uses open-source npm libraries as infrastructure:

- `mineflayer` is used to connect a bot to a Minecraft server, read nearby player entities, equip a sword, control movement, and perform attacks.
- `prismarine-physics` is used to simulate Minecraft-like player movement in the local duel environment.
- `minecraft-data` and `prismarine-block` are used to construct the flat local world needed by the physics simulator.
- `vec3` is used for three-dimensional position and velocity vectors.
- TypeScript and `tsx` are used for development and execution.

These libraries were not written by me. They provide the environment, Minecraft protocol interaction, block data, physics approximation, and vector primitives. The project does not copy a complete MCTS implementation from an external tutorial or repository.

AI-Assisted Work

AI assistance was used during development to help inspect the assignment brief, debug TypeScript code, improve the structure of the experiment generator, generate LaTeX report text, and improve readability. AI assistance also helped add and document the later experimental variations: hybrid MCTS with a heuristic action bonus, risk-aware reward shaping, high-exploration MCTS, and higher-budget MCTS.

The AI-assisted work should be understood as programming and writing assistance, not as an external source of experimental results. The reported results were generated by running the local code in this repository.

My Own Implementation Work

The project-specific implementation work consists of:

- Designing the Minecraft sword-duel problem and choosing MCTS as the main AI algorithm.
- Defining the combat action space: advancing, sprinting, jumping, strafing, retreating, holding position, and attacking.
- Implementing the simulated fighter state, local flat-world duel setup, attack cooldowns, health tracking, and duel loop.
- Implementing the MCTS decision process over combat actions, including tree nodes, UCT selection, rollout simulation, backpropagation, and action selection.
- Designing the reward function for survival, damage, spacing, cooldown timing, and attack windows.
- Designing the algorithm variations and comparing them experimentally.
- Creating the experiment logging pipeline that writes CSV, JSON, and SVG report materials.
- Interpreting the experimental results and writing the final report.

How to Identify the Main Code

The main source files are:

- `src/bot.ts`: runtime entry point, Mineflayer server mode, command-line configuration, and control application.
- `src/combat.ts`: combat state, local physics environment, MCTS planner, reward function, algorithm variations, and duel recording.
- `src/report-materials.ts`: repeated experiment generator, CSV summaries, raw JSON traces, and visual material generation.

Academic Integrity Declaration

To the best of my knowledge, this statement accurately describes the origin of the submitted work. Existing libraries are acknowledged above, AI-assisted work is disclosed, and the project-specific design, integration, experimentation, and interpretation are presented as my own work.